

Hoje vamos continuar aprendendo...

REACT:D

PRIMEIRA PÁGINA DA APLICAÇÃO

Começo

Vamos começar com a importação do Vite para nosso projeto usando o comando abaixo na pasta do projeto que estamos criando:

```
npm create vite@latest  
nomeDoNossoApp-- --template react
```

Mudando o “nomeDoNossoApp” pelo nome do aplicativo como desejar

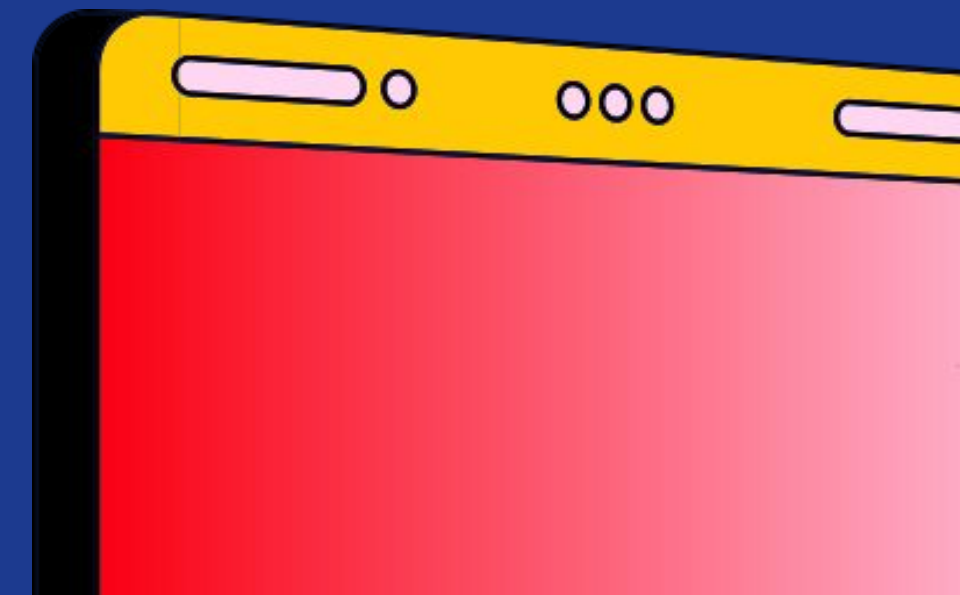
Iniciando

Após isso nós já temos a possibilidade de iniciar, podemos acessar a pasta diretamente pelo próprio prompt usando cd e colocando o nome da pasta que acabamos de criar e depois disso executar os dois comandos abaixo caso já não tenha iniciado o serviço do react:

- npm install
- npm run dev

Vínculo e início

Agora vamos fazer o vínculo do nosso projeto com o Visual Code e após isso vamos criar um novo arquivo chamado Login.jsx e vamos trabalhar nele.



React

Nós trabalhamos no arquivo .jsx, não JS.

Trabalhando em JSX nós usamos uma “extensão” do HTML no nosso arquivo JS, dessa forma temos uma sintaxe um pouco diferente do Java script normal.

Bora começar?

Início da aplicação

```
import { useState } from "react";
```

```
function Login() {  
  const [email, setEmail] = useState("");  
  const [senha, setSenha] = useState("");  
  const [erro, setErro] = useState("");  
  
  ...  
}
```

Import

O que basicamente o que o *import {useState} from "react"* faz é:

Ir até a biblioteca do REACT e chama a função *useState* é chamado de **Hook**. Ele permite que funções “lembrem” de informações entre uma renderização e outra.

No caso variáveis comuns não avisam a tela quando elas mudam, já com o `useState` isso resolve essa questão, onde... Se atualizarmos um contador de 0 para 1, ele irá mudar na tela sem a necessidade de realmente atualizar a tela com um F5.

Bora começar?

Início da aplicação

```
import { useState } from "react";  
  
function Login() {  
  const [email, setEmail] = useState("");  
  const [senha, setSenha] = useState("");  
  const [erro, setErro] = useState("");  
  
  ...  
}
```

function

Já o function Login é nossa função que será a base do nosso sistema, essa é como se fosse nossa classe, tudo que irá aparecer na tela nós vamos colocar nessa função, o interessante e estranho de quem vem de outras linguagens é que podemos **ANINHAR** funções...

Isso é uma prática bem específica, as funções internas elas definem comportamento da nossa aplicação;

Bora começar?

Início da aplicação

```
import { useState } from "react";  
  
function Login() {  
  const [email, setEmail] = useState("");  
  const [senha, setSenha] = useState("");  
  const [erro, setErro] = useState("");  
  
  ...  
}
```

const

As consts são nossas variáveis e vem de constante. Ela serve para declarar uma variável que, após receber um valor inicial, não pode ser "reatribuída".

Bora começar?

Início da aplicação

...

```
function handleSubmit(e) {  
  e.preventDefault();  
  
  if (!email || !senha) {  
    setErro("Preencha todos os campos.");  
    return;  
  }  
}
```

...

function handleSubmit(e)

Nossa primeira função aninhada, ela está dentro da nossa função de Login e ela tem uma.

O e que tem na declaração da nossa função, é uma convenção, ele basicamente significa que ali nós vamos usar eventos. Poderíamos inclusive mudar de e para eventos. Esse evento terá as informações de quem disparou o evento, que tipo de evento foi, posição do mouse e tecla pressionada.

Bora começar?

Início da aplicação

...

```
if (email === "user@email.com" && senha  
=== "123456") {  
  alert("Login realizado com sucesso!");  
  setErro("");  
} else {  
  setErro("E-mail ou senha incorretos.");  
}  
}
```

...

if / else

As condicionais if e else funcionam da mesma maneira que vemos em outras linguagens, nesse exemplo nós iremos dar um check se o e-mail introduzido é “[user@email.com](#)” e a senha é “123456” e caso der “match” vai funcionar e vai mudar e gerar um alert na página dizendo que o Login foi realizado, já ao contrário irá mudar a mensagem de erro para “E-mail ou senha incorretos”.

E assim finalizamos nossas duas funções, a aninhada e também a de Login.

Bora começar?

Início da aplicação

...

```
return (  
  <div className="container">  
    <form onSubmit={handleSubmit}>  
      <h2>Login</h2>  
    </form>  
  </div>  
)
```

...

return

O return no JS basicamente termina em uma linha só, por isso ele é usado em () e não chaves como funções/métodos/classes. Porém se precisarmos usar mais linha nós podemos usar seguindo a sintaxe do exemplo ao lado.

E dentro do return nós vamos trabalhar com trechos em HTML e também em JS.

Bora começar?

Início da aplicação

```
...  
<input  
  type="email"  
  placeholder="Email"  
  value={email}  
  onChange={(e) =>  
    setEmail(e.target.value)}  
  />  
...
```

input

quando usamos o Type = “email” nós dizemos para o navegador que esse campo é e-mail e o teclado do teu celular vai abrir no formato e-mail, por exemplo.

placeholder é aquele texto cinza que aparece, dá para mudar esse texto para o que for mais interessante.

value com o onChange deixa o input travado para ser somente leitura.

Bora começar?

Início da aplicação

```
...  
<input  
  type="password"  
  placeholder="Senha"  
  value={senha}  
  onChange={(e) =>  
setSenha(e.target.value)}  
  />  
  {erro && <p  
className="erro">{erro}</p>}  
...
```

input

Se fossemos ler as duas ultimas linhas, leriamos ela assim:

Se a variável erro for verdadeira (tiver algum texto), ENTÃO mostre o parágrafo <p> com a mensagem.

Ele usa uma lógica do JavaScript chamada Curto-Circuito para decidir se deve ou não mostrar algo na tela.

Bora começar?

Início da aplicação

```
...  
  <button  
type="submit">Entrar</button>  
  </form>  
</div>  
);  
}  
  
export default Login;  
...
```

Button

É idêntico ao button do HTML com algumas mudanças, basicamente ele vai avisar nos eventos que nós clicamos nesse botão.

export default Login

Essa linha no final do arquivo é o que permite que esse componente seja usado em outros lugares.

Iniciando o CSS

Agora vamos estilizar a nossa página, ela deve ser estilizada de acordo com as tuas preferencias, porém vamos usar nesse exemplo uma estética que acabei criando, porém para usar a estilização vamos fazer o seguinte passo-a-passo.

A

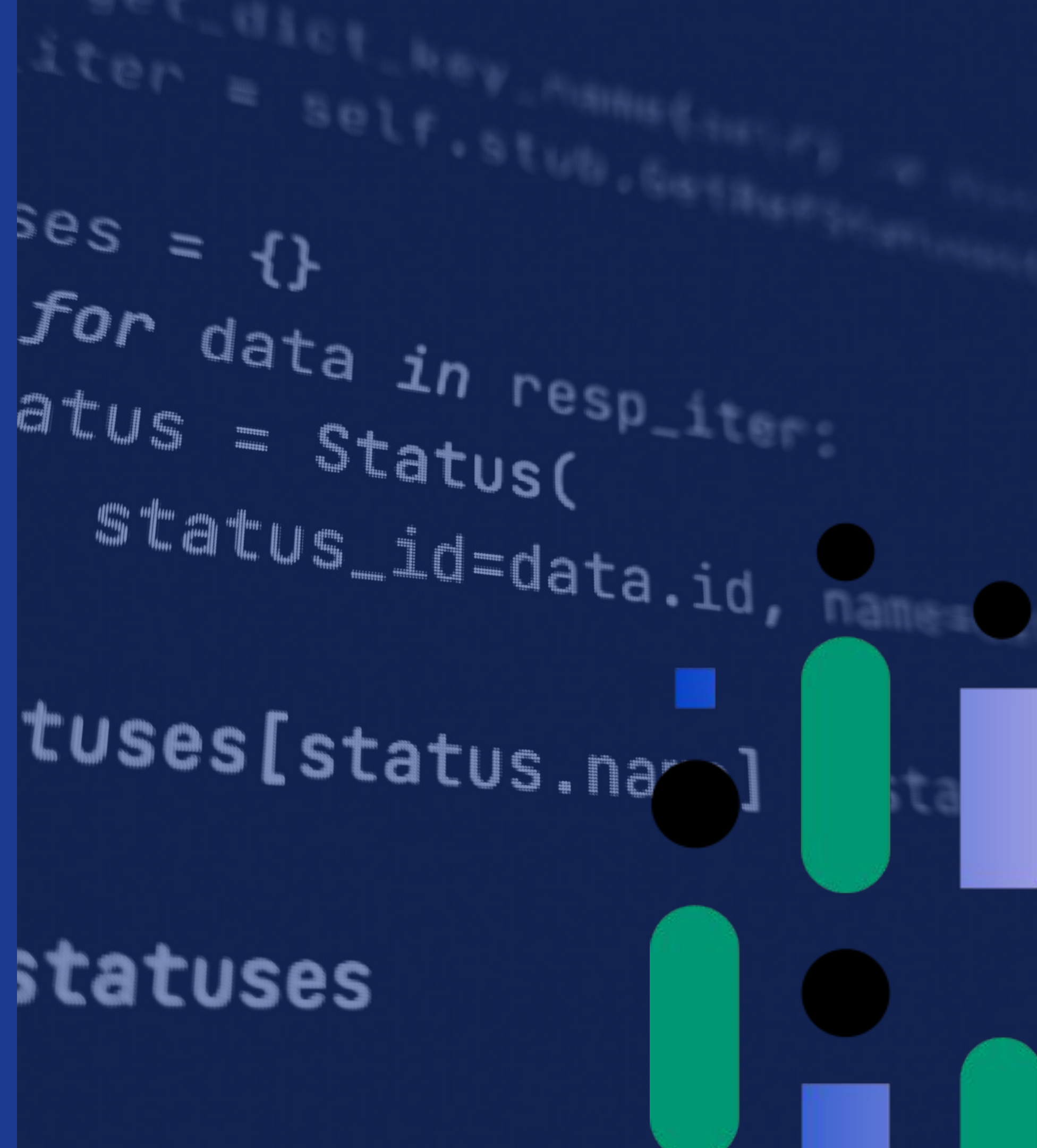
Criar um novo arquivo dentro do nosso projeto.

B

Nomear ele para Login.css .

C

Após isso vamos importar ele no nosso arquivo JSX usando o import `"/Login.css"`;



CSS

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  background: #f0f2f5;
}
form {
  background: white;
  padding: 2rem;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  display: flex;
  flex-direction: column;
  gap: 1rem;
  width: 300px;
}
input {
  padding: 0.75rem;
  border: 1px solid #ccc;
  border-radius: 6px;
  font-size: 1rem;
}
button {
  padding: 0.75rem;
  background: #4f46e5;
  color: white;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  cursor: pointer;
}
button:hover {
  background: #4338ca;
}
.erro {
  color: red;
  font-size: 0.85rem;
  margin: 0;
}
```

Por fim:

Importar o Login no arquivo App.JSX.

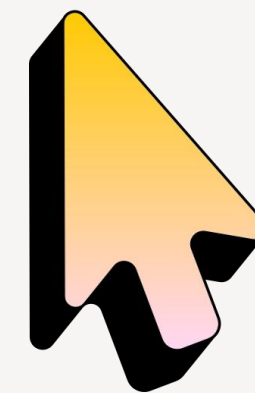
```
import Login from "./Login";
```

```
function App() {
```

```
  return <Login />;
```

```
}
```

```
export default App;
```



E agora:

- 01 Adicionar botão mostrar/ocular senha.
- 02 Editar a estética no CSS e deixar adequado a estética que você deseja para tua aplicação
- 03 Começar a criar a nova página da tua aplicação conforme conversaremos em aula :)

